Flowdesk - Test Specification, Traceability Matrix, and Technical Justification

Project: Flowdesk - Business Management System
Company / Team: Runtime Dynamics (Team 4)
Members: Cristopher Lanchimba, Cristopher Guañuña, Julián Jaramillo
Course: Object-Oriented Programming (NRC: 36862)
University: Universidad de las Fuerzas Armadas ESPE

1. Test Cases Documentation

Below are the formal test cases for the verification and validation of the Flowdesk system, detailing preconditions, inputs, execution steps, and expected results for each key function of the application.

CP-01: User Authentication and Role-Based Access Control

- **Functional Requirement:** RF-01 (Login and Access Control)
- **Classes Involved:** models.User, enums.Role, com.mycompany.flowdesk.Flowdesk
- **Precondition:** Application launched from Flowdesk.main().
- **Inputs:**
  - Selected Role: 1 (Role.MANAGER)
  - Username: admin
  - Password: 123
- **Execution Steps:**
  1. Run the application in the console.
  2. Select option 1 (Manager) in the main menu.
  3. Enter username "admin" and password "123" when prompted by User.authenticateFromConsole().
- **Expected Result:** Console message: "Sesion iniciada exitosamente para el usuario: admin" and display of the Manager-exclusive menu (Generate Reports, View Cash Register Status, Log Out).

CP-02: Customer Registration and Validation

- **Functional Requirement:** RF-02 (Customer Management)
- **Classes Involved:** models.Customer, utils.Validation, data.DataManager
- **Precondition:** Logged in with the EMPLOYEE role.
- **Inputs:**
  - Menu selection: 1 (Manage Customers) -> 1 (Register Customer)
  - Name: "Carlos Perez"
  - ID/Tax ID: "1723456789"
  - Phone: "0991234567"
  - Address: "Av. Amazonas 123"
- **Execution Steps:**

1. Access the Employee submenu and select "Manage Customers".
2. Choose the option "Register Customer".
3. Enter the required data via the console.

- **Expected Result:** Validation verifies non-empty fields and identification uniqueness. The system displays "Cliente registrado exitosamente" and adds the customer to DataManager.getAll().

CP-03: Product and Service Registration in the Catalog

- **Functional Requirement:** RF-03 (Product and Service Catalog)
- **Classes Involved:** models.CatalogItem, models.Product, models.Service, models.Category
- **Precondition:** Logged in with the EMPLOYEE role.
- **Inputs:**
  - Product: Name: "Teclado Mecanico", Price: 45.00, Stock: 10
  - Service: Name: "Mantenimiento PC", Price: 25.00
- **Execution Steps:**
  1. Select the option "Manage Products and Services".
  2. Register a Product by entering its attributes and stock.
  3. Register a Service by entering its details.
- **Expected Result:** The system validates prices and stock greater than zero. Polymorphic entities derived from CatalogItem are instantiated and saved to the global catalog.

CP-04: Work Order Creation and Subtotal Calculation

- **Functional Requirement:** RF-04 (Work Order Management)
- **Classes Involved:** models.WorkOrder, models.OrderDetail, enums.OrderStatus
- **Precondition:** At least one customer ("Carlos Perez") and one catalog product ("Teclado Mecanico") exist.
- **Inputs:** Customer ID: "1723456789", Item: "Teclado Mecanico", Quantity: 2, Unit Price: 45.00
- **Execution Steps:**
  1. Select "Create New Work Order".
  2. Associate the order with the customer's ID.
  3. Add an OrderDetail specifying quantity and product.
  4. Confirm and save the order.
- **Expected Result:** OrderDetail calculates subtotal ($2 x 45.00 = 90.00). WorkOrder assigns total = 90.00, balance = 90.00, and initial status OrderStatus.WAITING.

CP-05: Payment Processing and Cash Register Movement

- **Functional Requirement:** RF-05 (Cash Register and Collections Management)
- **Classes Involved:** models.Payment, models.CashMovement, models.CashRegister, enums.PaymentMethod
- **Precondition:** Session started with CASHIER role and an existing order with a balance due (90.00).
- **Inputs:** Order ID: 1, Payment Amount: 90.00, Method: PaymentMethod.CASH
- **Execution Steps:**
  1. Log in as Cashier and choose "Register Order Payment".
  2. Select Order ID 1.

3. Enter the amount 90.00 and select cash as payment method.
- **Expected Result:** A Payment instance is created, the order's pending balance updates to 0.00, and an INCOME type CashMovement is automatically generated in CashRegister.

CP-06: Exporting Reports to CSV Format

- **Functional Requirement:** RF-06 (Report Generation and Export)
- **Classes Involved:** reports.ReportGenerator, reports.CsvExporter, reports.Report
- **Precondition:** Session started with MANAGER role and transactions previously recorded.
- **Inputs:** Menu option: 1 (Generate Report)
- **Execution Steps:**
  1. Log in as Manager.
  2. Execute the option "Generate Report" in the menu.
- **Expected Result:** ReportGenerator queries DataManager, and CsvExporter physically generates .csv files in the reportes/ directory with valid headers and records.

CP-07: Data Persistence and Recovery in JSON

- **Non-Functional Requirement:** RNF-01 (Disk Data Persistence)
- **Classes Involved:** data.DataManager, exceptions.PersistenceException
- **Precondition:** Test records created during the active session.
- **Inputs:** Menu option: 4 (Exit) to trigger the final save.
- **Execution Steps:**
  1. Select the option to exit the system.
  2. Restart the application using Flowdesk.main().
- **Expected Result:** DataManager.saveAll() saves state to data/users.json, and upon restart, DataManager.loadAll() restores all objects without throwing persistence errors.

2. Requirements Traceability Matrix

The following matrix establishes direct traceability between each Functional Requirement, its implementation in Java classes, its corresponding test case, and its acceptance criterion.

Requirement ID	Requirement Description	Java Module / Classes	Test Case	Acceptance Criterion
RF-01	User login and access control based on assigned role (Manager, Cashier, Employee).	models.User, enums.Role, Flowdesk.java	CP-01	Restricted access and authorized navigation based on user role.
RF-02	Register, search, and list print shop customers.	models.Custom er, utils.Validation, data.DataManag	CP-02	Mandatory fields validation and registration in persistent list.

Requirement ID	Requirement Description	Java Module / Classes	Test Case	Acceptance Criterion
		er		
RF-03	Manage the product and service catalog with categories.	models.CatalogItem, models.Product, models.Service, models.Category	CP-03	Polymorphic creation of items with positive prices and stock.
RF-04	Create and update Work Orders linking customers and items.	models.WorkOrder, models.OrderDetail, enums.OrderStatus	CP-04	Automatic calculation of subtotals, totals, and pending balance.
RF-05	Register order payments and cash register movements (income/expenses).	models.CashRegister, models.CashMovement, models.Payment, enums.PaymentMethod	CP-05	Balance updated to zero and total cash register balance incremented.
RF-06	Generate and export operational reports to CSV files.	reports.ReportGenerator, reports.CsvExporter, reports.Report	CP-06	Physical creation of readable .csv files in the reportes/ folder.

3. Technical Justification of Data Structures and Use of ArrayList

This section provides the technical rationale for choosing the dynamic structure java.util.ArrayList and formally documents the clarification regarding the requirement for static primitive arrays.

3.1. Technical Rationale for using java.util.ArrayList

- **Adaptive Dynamic Sizing:** In the operational context of a print shop, the volume of customers, products, work orders, and payments cannot be predicted at compile time. Using fixed-size static arrays would impose rigid limits and require costly manual array resizing operations.

- **JSON Persistence Compatibility:** Collections based on ArrayList interact naturally with the persistence layer (DataManager.java), facilitating the serialization and deserialization of object lists to and from JSON files.
- **Encapsulation and OOP Best Practices:** The ArrayList API provides optimized native methods such as add(), remove(), get(), contains(), and size(), which reduce index out-of-bounds errors and improve code maintainability.

3.2. Clarification Record Regarding Primitive Arrays Requirement

Explicit record is made of the observation noted in the domain objects specification document (*List of Possible Objects / ObjectList.pdf*):

"The prototype uses structures like ArrayList<Customer>, ArrayList<WorkOrder>, and ArrayList<Payment> to store dynamically sized groups of business objects. When primitive values need to be stored in collections, they are wrapped (e.g., ArrayList<Integer> and ArrayList<Double>). It is important to note that an ArrayList contains wrapper objects, not primitive arrays, and therefore does not automatically satisfy a requirement specifically asking for basic primitive array implementation."

Final Conclusion: Although the prototype uses dynamic collections and wrapper objects instead of static primitive arrays, this architectural choice is fully justified to ensure scalability, compatibility with the JSON persistence module, and compliance with Object-Oriented Programming design principles.